

# DNS + Anycast Hybrid Routing

## Proof-of-Concept Documentation

---

### QUIC Server-Side Migration for Anycast Connection Stability

PoC Implementation Guide • July 2026

This document describes a proof-of-concept implementation that combines DNS-based GeoDNS resolution with simulated anycast routing and QUIC server-side migration. The PoC demonstrates how QUIC's preferred\_address mechanism (RFC 9000, Section 9.6) can solve the fundamental instability problem of anycast-routed connections: when BGP route changes redirect traffic to a different Point of Presence (PoP), existing connections break. By migrating connections from the anycast PoP to a unicast backend server immediately after the TLS handshake, the connection becomes immune to subsequent anycast route changes. This two-phase routing approach combines the fast initial routing of anycast with the stability of unicast for connection lifetime.

# 1. Overview

---

## Problem: Anycast Route Instability

Anycast is a widely deployed routing technique where multiple servers share the same IP address, and BGP routing directs clients to the nearest server. While this provides excellent initial latency, it introduces a critical instability: when BGP routes change (due to network failures, congestion-based rerouting, or routine maintenance), existing connections are silently redirected to a different PoP that has no knowledge of the connection state. The result is an immediate, unrecoverable connection failure.

- Anycast routing can cause connection disruption during BGP route changes
- The new PoP receives packets for a connection it never established
- TCP connections break silently; retransmission timers eventually expire
- Even with connection ID-based load balancing, cross-PoP state sharing is impractical at scale

## Solution: QUIC Migration to Unicast Backend

QUIC server-side migration provides an elegant solution. The anycast PoP acts as the primary server, completing the TLS handshake and immediately advertising a `preferred_address` pointing to a unicast backend server. The client validates the new path and migrates. Once migration is complete, the connection runs directly between the client and the unicast backend, entirely bypassing the anycast routing layer.

- Use QUIC migration to move connections from anycast PoP to unicast backend
- After migration, the connection is immune to anycast route changes
- Two-phase routing: coarse-grained anycast for initial contact, fine-grained unicast for connection lifetime

**Key Insight: After migration, the connection is on a direct unicast path and is completely immune to anycast BGP route changes.**

# 2. Architecture

---

## Testbed Layout

The PoC uses our existing four-machine LAN testbed (141.217.168.x subnet) with the following role assignments:

| Role                | Machine         | IP Address      | Description                                          |
|---------------------|-----------------|-----------------|------------------------------------------------------|
| VIP (Anycast)       | Client loopback | 10.99.99.1      | Simulated anycast address (loopback + iptables DNAT) |
| PoP (Primary)       | opti7040        | 141.217.168.152 | Simulated nearest anycast Point of Presence          |
| Backend (Preferred) | homeserver2     | 141.217.168.143 | Final unicast destination after migration            |
| Alt PoP             | Proxmox VM      | 141.217.168.200 | Destination after simulated BGP route change         |
| Client              | optiplex7010    | 141.217.168.127 | QUIC client (neqo-client or Firefox)                 |

## Connection Flow

The connection proceeds through three distinct phases:

- **Phase 1 (Anycast):** Client connects to VIP 10.99.99.1:4433. iptables DNAT redirects traffic to primary server at .152 (simulates anycast nearest-PoP routing).
- **Phase 2 (Handshake + Migration):** Primary completes QUIC handshake, advertises preferred\_address = 141.217.168.143:4433. Client sends PATH\_CHALLENGE to .143.
- **Phase 3 (Unicast):** Client validates path to .143, migrates. Connection now runs directly client <-> .143 on a stable unicast path.

## 3. How Anycast Simulation Works

---

### iptables DNAT Redirection

Since we cannot deploy actual BGP anycast on a LAN, we simulate it using Linux network namespaces and iptables DNAT rules on the client machine. A virtual IP (VIP) 10.99.99.1 is assigned to the loopback interface, and iptables OUTPUT chain rules redirect all traffic destined for the VIP to the actual primary server IP.

- iptables DNAT redirects traffic from VIP (10.99.99.1) to primary (.152), simulating anycast routing to nearest PoP
- Primary completes QUIC handshake and advertises preferred\_address = .143
- Client migrates to .143 (direct unicast path, bypasses DNAT entirely)
- Changing the DNAT rule (simulating a BGP flap) has NO effect on the migrated connection

### DNS Resolution via dnsmasq

A local dnsmasq instance resolves a test domain (e.g., cdn.quic-test.local) to the primary server IP. This simulates GeoDNS behavior where a CDN's authoritative DNS returns the IP of the nearest anycast PoP. In a production deployment, this would be a real GeoDNS service returning the anycast VIP.

- dnsmasq resolves test domain to the primary IP (simulates GeoDNS)
- Client resolves domain, connects to VIP, DNAT routes to primary

**Note: The DNAT simulation is client-side only. In production, anycast routing happens at the network layer via BGP, requiring no client-side configuration.**

## 4. Scripts

---

The PoC includes four shell scripts that manage the simulation environment:

| Script               | Purpose                                                                                                                          | Requires sudo |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------|
| setup_anycast_sim.sh | Creates the VIP (10.99.99.1) on loopback, adds iptables DNAT rules to redirect VIP traffic to the primary server (.152)          | Yes           |
| setup_dnsmasq.sh     | Configures a local dnsmasq instance to resolve the test domain (cdn.quic-test.local) to the primary server IP, simulating GeoDNS | Yes           |

| Script             | Purpose                                                                                                                                               | Requires sudo |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| test_route_flap.sh | Demonstrates connection survival during a simulated BGP route change. Changes the DNAT target from .152 to .200 while a migrated connection is active | Yes           |
| teardown.sh        | Removes all iptables rules, deletes the VIP from loopback, stops dnsmasq, and restores the original DNS configuration                                 | Yes           |

## 5. How to Run

---

### Step 1: Setup (on client machine)

```
sudo ./setup_anycast_sim.sh
sudo ./setup_dnsmasq.sh
```

### Step 2: Start servers (on remote machines)

On **homeserver2** (preferred/backend server):

```
preferred-server 141.217.168.143:4433 9999
```

On **opti7040** (primary/PoP server):

```
primary-server 0.0.0.0:4433 141.217.168.143:4433 141.217.168.143:9999
```

### Step 3: Test connection through VIP

```
neqo-client https://10.99.99.1:4433/
```

The client connects to the VIP, which is DNAT'd to the primary. The primary completes the handshake and advertises the preferred address. The client migrates to the backend, and the connection proceeds on a direct unicast path.

### Step 4: Route flap test

```
sudo ./test_route_flap.sh
```

This script changes the DNAT target from .152 to .200 while a connection is active, demonstrating that migrated connections survive the route change.

### Step 5: Cleanup

```
sudo ./teardown.sh
```

## 6. Route Flap Test

---

The route flap test is the core demonstration of the PoC. It proves that QUIC migration makes connections resilient to anycast route changes.

### Test Procedure

- **Step 1:** Establish connection through VIP → primary (.152) → migration → preferred (.143). Connection is now on unicast path.
- **Step 2:** Change DNAT rule: VIP (10.99.99.1) now routes to .200 instead of .152. This simulates a BGP route change that would redirect anycast traffic to a different PoP.
- **Step 3:** Verify that the **migrated connection SURVIVES**. It is on the direct unicast path to .143 and is unaffected by the DNAT change.
- **Step 4:** Establish a **new** connection through VIP. It goes to .200 (the new PoP), demonstrating that the route change did take effect for new connections.

**Result: The migrated connection survives the simulated BGP route change. New connections go to the new PoP (.200), but existing migrated connections remain stable on their unicast backend (.143).**

## 7. Key Insight

---

### Traditional Anycast Limitations

In traditional anycast deployments, BGP route changes break existing connections because the new PoP has no state for the connection. This is a fundamental limitation of anycast routing: it optimizes for initial connection establishment but provides no stability guarantees for connection lifetime.

**Traditional anycast: BGP changes break existing connections with no recovery mechanism.**

### Two-Phase Routing with QUIC Migration

QUIC server-side migration introduces a two-phase routing model:

- **Phase 1 (Coarse-grained anycast):** Initial contact uses anycast for fast, proximity-based routing to the nearest PoP. This phase is brief (one RTT for handshake).
- **Phase 2 (Fine-grained unicast):** Connection migrates to a dedicated unicast backend. The connection is now "pinned" to this backend for its entire lifetime, immune to anycast routing changes.

### Production Applicability

This pattern is directly applicable to several large-scale deployment scenarios:

- **CDNs:** Edge PoPs use anycast for client proximity; migration moves long-lived connections (streaming, WebSocket) to stable origin or mid-tier servers
- **Edge computing:** Anycast routes to nearest edge node; migration moves stateful workloads to the optimal compute backend
- **Globally distributed services:** Anycast provides fast DNS-like routing; migration provides connection pinning without DNS TTL delays

**With QUIC migration, the connection is "pinned" to a unicast backend after handshake -- combining fast anycast initial routing with rock-solid unicast connection stability.**

## 8. Comparison

The following table compares three approaches to routing clients to distributed servers, highlighting the advantages of combining QUIC migration with anycast:

| Scenario             | Pure Anycast                      | DNS-based                            | QUIC Migration + Anycast          |
|----------------------|-----------------------------------|--------------------------------------|-----------------------------------|
| Initial routing      | BGP (fast, automatic)             | DNS TTL (slow, cached)               | BGP + migration (fast)            |
| Route change impact  | Connection breaks immediately     | Wait for DNS TTL to expire           | No impact on existing connections |
| Connection stability | Poor (any BGP flap breaks it)     | Medium (stable until DNS re-resolve) | Excellent (pinned to unicast)     |
| Data path            | Through PoP always (extra hop)    | Direct after DNS resolution          | Direct after migration (~1 RTT)   |
| Failover speed       | BGP convergence (seconds-minutes) | DNS TTL (seconds-hours)              | Instant (already on unicast)      |
| Client complexity    | None                              | DNS resolver logic                   | QUIC stack support (RFC 9000)     |

**QUIC Migration + Anycast achieves the best of both worlds: the fast initial routing of anycast with the connection stability of direct unicast paths. The migration overhead is minimal (one additional RTT for PATH\_CHALLENGE/PATH\_RESPONSE), and the resulting connection is immune to all subsequent routing changes.**